

Guía de Despliegue con Git

Para cuando quieras automatizar la subida de cambios al servidor — 3 de septiembre de 2026

Importante: Esta guía queda lista para cuando decidas usarla. No hace falta hacer nada de esto ahora ni cambia nada de lo que ya está funcionando.

¿Para qué sirve esto?

Hoy, para llevar un cambio de código al servidor hay que copiar archivos a mano de la PC de Windows al servidor de Ubuntu. Con Git, ese paso se reemplaza por dos comandos: uno para "subir" los cambios desde Windows y otro para "bajarlos" en el servidor. Además queda guardado un historial completo de todos los cambios hechos, por si alguna vez hay que volver atrás.

Paso 1 — Preparar el repositorio en Windows

Se ejecuta en la carpeta del proyecto, en la PC de Windows (si ya se corrió "git init" antes, este paso se puede omitir):

```
cd "C:\Users\Gaston C\fontana_movimientos"
git init
git add .
git commit -m "Version inicial"
```

Nota: El proyecto ya tiene un archivo .gitignore que excluye el archivo .env, la carpeta venv y la carpeta assets. Eso es intencional: son cosas que dependen de cada máquina y no se comparten por Git.

Paso 2 — Crear un repositorio remoto privado

Hace falta una cuenta en un servicio como GitHub. Ahí se crea un repositorio nuevo, marcado como "Private" (privado, para que nadie más lo vea), y SIN inicializarlo con un README (porque la carpeta local ya tiene contenido propio). El sitio va a mostrar la dirección del repositorio, algo como:

```
https://github.com/tu-usuario/fontana_movimientos.git
```

Paso 3 — Conectar y subir desde Windows

```
git remote add origin https://github.com/tu-usuario/fontana_movimientos.git
git branch -M main
git push -u origin main
```

Nota: La primera vez, GitHub va a pedir iniciar sesión. Ya no acepta la contraseña de la cuenta directamente: pide un "personal access token" (se genera desde la configuración de GitHub) o usar GitHub Desktop, que maneja el login solo.

Paso 4 — Primera vez en el servidor Ubuntu

En el servidor, en lugar de copiar la carpeta a mano, se descarga ("clona") el repositorio:

```
cd ~
git clone https://github.com/tu-usuario/fontana_movimientos.git
cd fontana_movimientos
python3.12 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Como .env, venv y assets no viajan por Git (a propósito), hay que recrearlos en el servidor:

- Crear el archivo .env a mano, con los mismos datos que ya usa el servidor (base de datos, DJANGO_SECRET_KEY, etc.).
- Generar la carpeta assets con collectstatic (ver más abajo).

```
python manage.py migrate
python manage.py createsuperuser # si todavía no hay usuarios cargados
python manage.py collectstatic --noinput
```

Nota: Si en el servidor ya existe una copia del proyecto (la que se llevó copiando archivos a mano), lo más prolijo es clonar el repositorio en una carpeta nueva, pasar el .env viejo a la carpeta nueva, y recién ahí apuntar el servicio systemd a la carpeta nueva. Si preferís hacerlo de otra manera, para no arriesgar nada del servidor ya andando, avisame y lo vemos juntos paso a paso.

Paso 5 — Rutina del día a día

En Windows, después de hacer cambios en el código:

```
git add .
git commit -m "Descripción breve del cambio"
git push
```

En Ubuntu, para traer esos cambios al servidor:

```
cd ~/fontana_movimientos
source venv/bin/activate
git pull
pip install -r requirements.txt
python manage.py migrate
python manage.py collectstatic --noinput
sudo systemctl restart fontana
```

Cosas para tener en cuenta

- Nunca subir el archivo .env a Git: tiene contraseñas y la clave secreta de Django. Ya está excluido por el .gitignore, así que no debería subirse por accidente.
- No editar archivos directamente en el servidor Ubuntu: si se edita ahí y después se hace git pull, puede generar conflictos. Todos los cambios de código conviene hacerlos siempre en Windows y bajarlos con git pull.
- pip install -r requirements.txt conviene correrlo en Ubuntu después de cada pull, por si el cambio agregó alguna librería nueva (no hace daño correrlo aunque no haya cambiado nada).